



TECHNICAL WHITEPAPER

SigniQ

Formal models of the six-state QKD layer, the teleportation-based QDS protocol, the statistical threat-detection engine, and the seeded evaluation.

TEAM PROMETHEUS · SIH26141

Quantum-secured document signatures · classical software simulation · 2026-09-29

Abstract

SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware

SigniQ is a classical software simulation of a quantum-secured document pipeline that implements a teleportation-based quantum digital signature (QDS) protocol together with a threat-detection layer built exclusively on quantum principles and statistics — Pauli eigenstate preparation, projective measurement, and closed-form concentration bounds — with no artificial intelligence or machine learning components of any kind. Signatures are constructed from genuine Bell-measurement outcomes produced by a statevector teleportation engine; whole-signature forgery requires guessing every Bell outcome, giving a forgery probability of $4^{-(q\lambda)}$ ($\approx 1.5 \times 10^{-5}$ at default parameters, tunable to $\approx 2.9 \times 10^{-39}$ at $q=16, \lambda=4$). Documents are sealed in AES-256-GCM `.qsig` containers and delivered peer-to-peer or through k-of-m consensus rings, with every security-relevant event appended to a hash-chained Merkle audit ledger with inclusion proofs. In seeded attack batteries over 120 trials per scheme, the system achieves 100% verification accuracy, 100% detection rate, and a 0.0% false-accept rate across five attack classes (forgery, impersonation, replay, channel tampering, unauthorized verification), while a simulated CHSH Bell test certifies the honest channel at $S = 2.847 \pm 0.007$. This document maps each claim to its implementing source; all quantum behavior is software simulation, and all results are measurements of the implemented model rather than statements about physical devices.

Contents

1. Introduction
 2. System architecture
 3. The six-state QKD layer
 4. Teleportation-based quantum digital signatures
 5. The threat-detection layer
 6. Document sealing and the audit ledger
 7. Evaluation
 8. Limitations and scope
 9. References
-

1. Introduction

1.1 Problem

Public-key signatures (RSA, ECDSA) rest on hardness assumptions that Shor's algorithm breaks on a large quantum computer. Quantum digital signatures replace those assumptions with information-theoretic ones: security follows from the physics of measurement, not computational difficulty. The SIH26141 problem statement asks for a *simulation* of a teleportation-based QDS protocol with a statistical threat-detection layer that detects forgery, impersonation, replay, and channel manipulation, reports detection rates and

false-accept rates from measurement statistics, and preserves the protocol's information-theoretic security guarantees — explicitly without AI or ML.

SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware

1.2 Approach

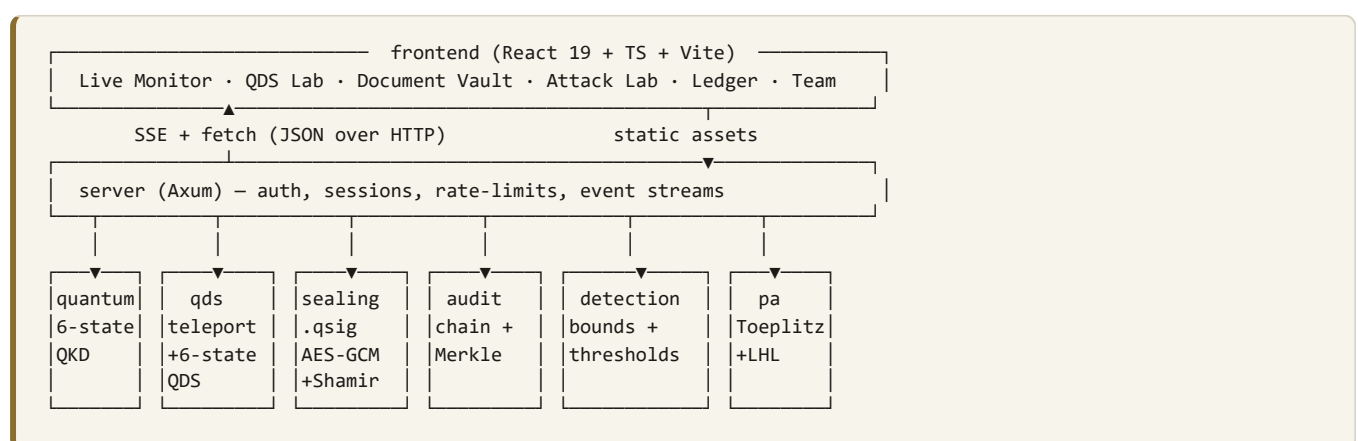
SigniQ implements the full pipeline in a Rust workspace with a React dashboard:

- **Keys** from a six-state QKD simulation (Bruß 1998), with Cascade/Winnow error reconciliation and Toeplitz universal₂ privacy amplification sized by the leftover hash lemma.
- **Signatures** from a teleportation-based QDS (Gottesman–Chuang 2001 verdict semantics; Weng et al. 2021 six-state messaging), every bit backed by Born-rule Bell-measurement outcomes.
- **Detection** from Hoeffding/Chernoff bounds and exact binomial tails over measured statistics — every threshold inspectable, no learned parameters.
- **Documents** sealed in AES-256-GCM containers, bound to QDS-derived keys, audited in an RFC 6962-discipline Merkle ledger.

1.3 Contributions

1. A clause-by-clause implementation of the problem statement, mapped to source in `docs/DELIVERABLES.md`.
2. A statevector teleportation engine whose per-bit correction record is the signature (7 physics tests including Born-rule statistics and fidelity 1.0).
3. A temporal trap binding each signature to a notary's Lamport clock, entropy salt, and hash-chain link, making replay a *sequence* failure rather than a lookup miss.
4. A reproducible, seeded evaluation harness (`scripts/smoke-isolated.mjs` , `/api/qds/metrics`) with confusion-matrix metrics including an explicit false-accept rate.

2. System architecture



Crate responsibilities. `quantum` models six-state preparation/transmission (Pauli eigenstates, basis-choice collapse, environmental noise as an independent channel from intercept-resend), relays, decoy states, and the CHSH Bell test. `pa` implements the Toeplitz extractor and entropy budget. `detection` implements Hoeffding/Chernoff bounds, exact binomial tails, dynamic thresholds, and finite-key settlement. `qds`

implements both signature schemes (teleportation-based and six-state), the attack battery, the metrics engine, and the temporal trap. `sealing` owns the `.qsig` container format. `audit` owns the hash chain, Merkle root, and inclusion proofs. `server` binds them behind the REST/SSE API; `trust` and `main_app` provide terminal front-ends.

Trust boundaries. Raw session key material never leaves the server; the browser receives only key *commitments*. The audit ledger's integrity is verifiable by anyone, but its publisher identity is not proven by the ledger itself (§6.3). The signing notary ("Trent") holds per-session secrets in memory with replay-state snapshotted to disk so restarts never re-arm consumed signatures.

3. The six-state QKD layer

3.1 Preparation and sifting

Alice prepares each qubit in one of six Pauli eigenstates — `PauliBasis` $\in \{X, Y, Z\} \times \text{PauliState} \in \{\text{Positive}, \text{Negative}\}$ — uniformly at random (`quantum/src/lib.rs::generate_eigenstates`). Bob measures in a random basis; positions where the bases match are sifted. With three bases the sifting yield is $\approx 1/3$ (measured: 20,000 qubits $\rightarrow$ 6,620 sifted). Bob's measurement is a projective measurement in his declared basis (`qds/src/six_state.rs::projective_measure`), collapsing the state per the Born rule.

3.2 Channel model and QBER

Environmental noise (fiber attenuation/turbulence) is modeled as bit flips *in flight*, independent of basis; an eavesdropper intercepting fraction f of the traffic (measure-in-random-basis, resend) is wrong about the basis $2/3$ of the time and then wrong about the bit half of that, giving $\text{QBER} \rightarrow f/3 = 33.3\%$ under full interception (measured: 33.60% at $f = 1.0$, §7.2). Noise and interception contribute independently, which the sweep endpoint exploits to separate "degraded" from "under attack."

3.3 Finite-key statistics

The decision threshold is not a constant. For n sifted samples and confidence $1-\delta$, the Hoeffding bound gives $\epsilon(n, \delta) = \sqrt{(\ln(2/\delta)/2n)}$; the rejection line is the noise floor plus $\epsilon(n)$, narrowing like a vice as n grows (`detection/src/bounds.rs`). A verdict is only issued when the sample is statistically *settled* (`finite_key_ok` : $\epsilon(n)$ smaller than the gap between decision lines). Channel classification is three-way — secure / degraded / under attack — and a classified-under-attack channel aborts key distillation outright.

3.4 Reconciliation and privacy amplification

Noisy-but-secure keys are repaired rather than discarded: an opening block sized $\approx 0.73/\text{QBER}$ (Cascade scheduling, Brassard–Salvail 1993), Hamming(7,4) syndromes (Winnow, Buttlar et al. 2003) in pass one, then Cascade bisection with doubling blocks over re-permuted orders until a pass finds nothing. Every disclosed parity bit is priced into `leakage_bits` . Privacy amplification then applies a Toeplitz universal₂ extractor (`pa/src/lib.rs`): the output length m is chosen so the leftover hash lemma caps Eve's mutual information at a target ϵ , charging intercepted positions (each burns one bit of min-entropy under the standard six-state accounting, `eve_information_fraction` = $\text{QBER}/3$ -scaled) and reconciliation leakage. The

achieved ϵ is reported with the key (`SecurityAccounting` in the run response). A conservative presentation floor of 2^{-300} bounds displayed epsilon values.

SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware

3.5 Decoy states and the Bell test

A decoy-state layer (Hwang 2003; Lo–Ma–Chen 2005) models photon-number statistics and flags photon-number-splitting via the per-photon efficiency ratio $\eta_{\text{signal}}/\eta_{\text{decoy}} > 2$ (`estimate_pns`); it is a detection layer and does not feed key generation. Independently, every run executes a CHSH Bell test (`quantum/src/chsh.rs`): entangled pairs measured in the canonical geometry, S computed from outcome correlations with a finite-sample error bar. An honest channel certifies at $S \approx 2.83\text{--}2.85$ (measured: $S = 2.847 \pm 0.007$ over 160,000 rounds, margin 120.7σ , §7.2); an intercept-resend eavesdropper — even one undiscovered by QBER tuning — caps $S \leq 2$ because measurement destroys the entangled correlations. QBER is the statistical detector; the Bell test is the physical one.

4. Teleportation-based quantum digital signatures

4.1 The teleportation engine

`qds/src/teleport.rs` implements a genuine 3-qubit complex-amplitude statevector: EPR pair preparation (Hadamard $\otimes$ CNOT), the standard Bell-measurement circuit (CNOT + Hadamard + projective measurement), Born-rule collapse — nothing hard-coded — and conditional Pauli corrections (I / X / Z / ZX) restoring Bob's state with fidelity 1.0 (verified for basis states, $\pm$ states, i-states, and arbitrary complex superpositions). Alice's Bell outcome — two classical bits — cannot be biased: the test

`measurement_statistics_follow_the_born_rule` asserts each of the four outcomes lands at $\approx 25\%$.

4.2 Signature construction and verification

The signer ("Alice") derives the signature's correction-bit sequence from actual teleportation outcomes of the message bits. The notary ("Trent," the Weng-et-al network center role) runs `Trent::setup` — per-session qubit count q , repetition λ , key material — issues single-use nonces, and records the signature ledger. Verification (`qds::verify`) applies projective measurement rules to the correction statistics and returns one of three verdicts per Gottesman–Chuang semantics: **1-ACC** (unconditional accept), **0-ACC** (accept for Alice $\leftrightarrow$ Bob but not transferable), **REJ** (reject). A second-verifier transferability check ("Charlie consensus") grades whether a signature survives forwarding.

4.3 Forgery probability

An attacker who guesses the correction bits at every one of the $q \cdot \lambda$ signature positions succeeds with probability $(1/4)^{(q \cdot \lambda)}$:

$$P_{\{\text{forge}\}} = 4^{-q \cdot \lambda}$$

implemented exactly as `theory_forgery_probability(qubit_count, lambda) = 0.25^(qubit_count * lambda)`

(`qds/src/lib.rs` ~L988). At defaults ($q=8, \lambda=1$) this is $4^{-8} \approx 1.53 \times 10^{-5}$; at (16, 4) it is $4^{-64} \approx 2.9 \times 10^{-39}$. The empirical Monte-Carlo estimate (`estimate_forgery_probability`) measured 0 successes in 120 trials at (8, 1)

(§7.1), consistent with the unit-suite assertion and with a 20,000-trial validation at (1,1) matching 0.25 exactly on earlier builds.

SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware

4.4 The temporal trap (replay defense)

Each signature carries a quantum-entropy timestamp (`qds/src/temporal.rs`): the notary's sifting-window sequence number (a Lamport logical clock advanced only on acceptance), an entropy salt distilled from the signing teleportations' Bell outcomes, a hash-chain link $H(\text{prev} \parallel \text{window} \parallel \text{entropy})$, and a tag welding the timestamp to the exact correction bits. Verification enforces four checks — tag (transplant), chain (forged timestamps), consumed-window sequence (replay), and a freshness horizon — so a replayed signature fails a *sequence* test ("signature is from the past"), and a forged future timestamp fails the chain check. The trap's memory persists across restarts via a snapshot file (`qds_events.replay-state.json` -derived path), so nothing re-arms.

4.5 Consensus ring

For multiparty workflows (`qds/src/consensus.rs`), a signature is delivered to m receivers over channels with independent noise draws (a compromised member can be injected at any index); each member re-verifies with full trap semantics, and the ring aggregates an unlock gate ($\geq k$ accepts) and a transfer grade ($\geq k$ 1-ACC). Ring-wide forgery success is bounded by p^k (union bound). $k=3$ -of-5 tolerates one compromised member.

5. The threat-detection layer

5.1 No AI/ML — by construction

The detection layer contains no training loops, no learned parameters, and no ML dependencies (workspace-wide `Cargo.toml` `grep`: zero matches for `linfa/candle/tch/ndarray/smartcore/tensorflow/torch/sklearn/xgboost/onnx`). Every verdict is a threshold comparison or a closed-form bound applied to measured statistics — inspectable in one screenful of code. This is the problem statement's "explicitly without any AI or ML" implemented as an architectural property, not a styling choice.

5.2 Detection statistics

Seeded Monte-Carlo batteries (`qds/src/metrics.rs`) run each attack class against both signature schemes and tally a confusion matrix over controlled experiments:

Count	Meaning
<code>true_positives</code>	attacks correctly rejected/flagged
<code>false_negatives</code>	attacks wrongly accepted
<code>true_negatives</code>	legitimate signatures correctly accepted
<code>false_positives</code>	legitimate signatures wrongly flagged

From these, the module derives:

- **Detection rate** = $TP / (TP + FN)$ — recall over attack events (`detection_rate`), also broken out **per attack class** (`detection_by_class`, `ClassDetection::rate`).
- **False-accept rate** = $FN / (FN + TP)$ — the rate at which attacks are wrongly accepted (`false_negative_rate`). The field carries the classical "false-negative" name because the matrix treats "attack present" as the positive class; the quantity is exactly the problem statement's false-accept rate. We state the mapping rather than rename the field.
- **False-alarm rate** = $FP / (FP + TN)$ over legitimate events — how often honest signatures are rejected.

5.3 Attack coverage

Attack class	Simulation	Detection
Forgery	guessed correction bits under a fresh nonce (<code>attempt_forgery</code> , <code>attempt_six_state_forgery</code>)	session-commitment mismatch; $\approx 75\%$ bit mismatch $\Rightarrow$ REJ
Impersonation	genuine signature transplanted to a different message (<code>attempt_impersonation</code>)	message-hash binding fails at $\approx$ half of positions
Replay	accepted signature re-presented verbatim (<code>attempt_replay</code>)	temporal trap: consumed window $\Rightarrow$ sequence mismatch
Channel tampering	fraction-tampered channels (<code>attempt_channel_tampering</code> , QKD scenarios)	QBER vs dynamic threshold $\Rightarrow$ abort; mismatch-ratio flag
Unauthorized verification	verifier without session key material (<code>attempt_unauthorized_verification</code>)	no conclusive key material $\Rightarrow$ flagged before statistics

5.4 The confidence engine

`GET /api/stats/bounds` exposes the machinery the verdicts ride on: two-sided confidence intervals, the N-scaling threshold curve, REJECT-verdict p-values $P(X \geq k \mid \text{honest at threshold})$ with qualitative bands — including an explicit "**undersized**" band when the sample cannot separate honesty from attack, because a detector that knows when it cannot decide is more honest than one that always answers.

6. Document sealing and the audit ledger

6.1 The `.qsig` container

Sealing keys are derived from six-state QDS key-generation sessions (`server/src/qds_keys.rs`, `POST /api/doc/qds/key`) — the document layer runs on QDS, not raw QKD. The container (`sealing/src/lib.rs`) is JSON with a `QSIG1` magic: payload sealed under **AES-256-GCM** (v2) with container metadata bound as associated data (a swapped document with an intact body fails decryption), plus an independent HMAC-SHA256 tag; the current v3 envelope seals the entire container JSON so only the key commitment (a SHA-256 digest) and envelope nonce are visible on disk. Verification comparisons are constant-time; a Shamir k-of-m mode (GF(251)) splits the seal key so below-quorum recovery is cryptographically impossible.

6.2 Delivery workflows

Peer transfer works laptop-to-laptop (/api/doc/send to a peer URL) or same-server user-addressed; the consensus-ring workflow (/api/doc/ring/send, /api/doc/ring/attest) keeps every inbox item **locked until k attestations** — verified live: open refused at o/k ("CONSENSUS GATE LOCKED"), permitted at k with byte-exact recovery (§7.3). The attack theater (/api/doc/attack-theater) stages capture → inspect/tamper → forward → recipient rejection as a narrated replay recorded in the ledger.

6.3 The audit ledger

Every security-relevant event lands in a persistent JSONL ledger (audit/src/lib.rs): domain-separated leaves (AUD1 prefix, length-framed), RFC 6962-tagged interior nodes (the Merkle second-preimage defense), a Merkle root, per-event inclusion proofs, and chain verification (GET /api/audit/verify). The dashboard exports a self-contained bundle and verifies it offline in the browser (frontend/src/auditEvidence.ts). **Scope, stated precisely:** the ledger proves internal consistency — append-only ordering and no retro-editing. It does *not* authenticate the publisher; an expected root must arrive through an independent trusted channel for that. There is deliberately no key-revocation lifecycle: a QKD abort or a rejected signature is not revocation, and adding one casually would create misleading security semantics.

7. Evaluation

7.1 Methodology

All experiments run against an **isolated scratch instance** (env-var-segregated data files; real runtime data untouched) on the current build, with fixed seeds for reproducibility. QDS metrics: GET /api/qds/metrics? trials=120&seed=42 . QKD anchors: POST /api/run {key_length: 20000, seed: 424242} . End-to-end behavior: node scripts/smoke-isolated.mjs (47 checks). Timings are from this machine (debug build) and are indicative only.

7.2 Signature-scheme metrics (trials = 120, seed = 42)

Metric	Teleport QDS	Six-state QDS
Verification accuracy	100.0%	100.0%
Detection rate (TPR)	100.0%	100.0%
False-accept rate (missed attacks)	0.0%	0.0%
False-alarm rate (legitimate flagged)	0.0%	0.0%
Per-class detection (forgery / impersonation / replay / channel tampering / unauthorized)	100% ×5	100% ×5
Theoretical forgery probability $4^{(-q\lambda)}$	1.53×10^{-5} (q=8, λ=1)	—
Empirical forgery successes	0 / 120	0 / 120

Metric	Teleport QDS	Six-state QDS
Mean sign / verify time	41 μ s / 18 μ s	—

7.3 QKD channel anchors (20,000 qubits, seed 424242)

Scenario	Sifted	QBER	Classification	Key distilled
Secure channel	6,620	0.00%	secure	yes
Full intercept-resend (f = 1.0)	6,651	33.60% $\approx$ 1/3	under_attack	aborted

CHSH Bell test on the secure channel: **S = 2.847 $\pm$ 0.007** over 160,000 rounds — certified, margin 120.7 σ (theory: $2\sqrt{2} \approx 2.828$; classical bound: 2).

7.4 End-to-end behavior (47/47 checks)

The isolated smoke battery verifies, against live endpoints: account provisioning; identical seeded key distillation on two "laptops"; seal $\rightarrow$ deliver $\rightarrow$ verify with AES-GCM/HMAC/hash agreement; **all four tamper modes rejected with named causes** (GCM tag mismatch $\times 3$; key-commitment mismatch on reseal); **consensus ring locked at k-1** ("CONSENSUS GATE LOCKED — o/3 attestations") and **opened at quorum with byte-exact document recovery**; **all six QDS attack classes rejected** with named temporal/statistical reasons; notary clock monotonicity across signatures (replay-state persistence); metrics consistency; the bounds engine; QDS key derivation; and the audit ledger (chain intact, inclusion proof verified, 135 KB export).

7.5 Reproducibility

Every seeded experiment reproduces bit-identically given the same seed; unseeded runs use OS CSPRNG randomness. The full battery re-runs with the commands in the repository README's Verification section.

7.6 Worked example: the bounds engine on one measured batch

The confidence engine turns a raw measurement count into a defensible verdict in closed form. Running the main binary (`cargo run -p main_app` , seed 42) produces a real worked instance — full intercept-resend over 1,027 sifted positions, 331 mismatches:

Step	Computation	Result
Observed rate	$\hat{p} = k/n = 331/1027$	0.3223
Hoeffding slack	$\varepsilon = \sqrt{(\ln(2/\delta)/2n)}$, $\delta = 0.01$	0.0508
Two-sided CI	$\hat{p} \pm \varepsilon$	[0.2715, 0.3731]
Chernoff upper	$\hat{p}(1+\gamma)$, γ solved from $\exp(-\hat{p}^2\gamma^2/(2+\gamma)) = \delta/2$	0.3657 ($\times 1.02$ sharper than Hoeffding here)
Verdict p-value	$P(X \geq 331 \mid \text{honest at the 0.15 line})$, exact binomial tail	1.3×10^{-43} — Airtight

The same engine applied to the clean run (0 mismatches in 992) returns an interval of [0, 0.0517] and refuses to over-claim: the rejection p-value is 1.0 and the band is **Undersized** — at this mismatch count the

sample cannot separate honesty from attack, so the engine says so instead of pretending (the dynamic threshold has narrowed from 0.4269 at $n = 16$ to 0.0717 at $n = 992$). A detector that knows when it cannot decide is the honest baseline the problem statement's statistical posture demands.

7.7 The main binary as a reproducible demonstration

`cargo run -p main_app` walks every engine in one seeded pass: (A) secure six-state run $\rightarrow$ distillation $\rightarrow$ HMAC binding; (B) intercept-resend $\rightarrow$ QBER alarm $\rightarrow$ distillation aborted (stated as a rejected run, not key revocation); (C) Chernoff–Hoeffding dossiers for both channels (§7.6); (D) seal $\rightarrow$ v3-envelope round-trip $\rightarrow$ two tamper modes (envelope byte-flip rejected at GCM decryption; inner-payload corruption rejected at integrity) $\rightarrow$ audit chain verified + Merkle inclusion proof `verified=true`; (E) the full evaluation sweep of §7.2. It is the fastest end-to-end sanity check that the whole pipeline is wired, and its output is stable for a fixed seed.

8. Limitations and scope

1. **All quantum behavior is classical software simulation.** No quantum hardware, physical detector, or physical channel participates anywhere in the system.
2. **Simulation demonstrates statistics; it cannot certify devices.** The protocol's information-theoretic arguments are preserved in the model; no claim is made about physical deployments.
3. **The audit ledger proves internal consistency only**, not publisher identity (§6.3).
4. **No key-revocation lifecycle exists.**
5. **Individual-attack model.** The six-state layer models individual (intercept-resend class) attacks; coherent-attack security is analytic, not simulated. The CHSH layer models the fair-sampling regime (no detection loopholes); detector physics (dead time, afterpulsing, efficiency mismatch) is simplified.
6. **Idealized Bell-pair fidelity** in the teleportation engine; channel noise enters on the classical side (`qds::noisy`).
7. **Metrics are seeded-reproducible, not universal**; timings are machine-specific.
8. **Educational prototype.** Cryptographic constructions and deployment configuration require review before any production use; deployment itself is prepared but not performed.

9. References

1. D. Bruß, "Optimal Eavesdropping in Quantum Cryptography with Six States," *Phys. Rev. Lett.* **81**, 3018 (1998). arXiv:quant-ph/9805019.
2. C. H. Bennett, G. Brassard, C. Crépeau, U. M. Maurer, "Generalized Privacy Amplification," *IEEE Trans. Inf. Theory* **41**, 1915 (1995).
3. G. Brassard, L. Salvail, "Secret-Key Reconciliation by Public Discussion," *EUROCRYPT* (1993).
4. W. T. Buttler, S. K. Lamoreaux, et al., "Fast, Efficient Error Reconciliation for Quantum Cryptography," *Phys. Rev. A* **67**, 052303 (2003).

5. A. K. Ekert, "Quantum Cryptography Based on Bell's Theorem," *Phys. Rev. Lett.* **67**, 661 (1991).
6. A. Acín, N. Gisin, L. Masanes, "From Bell's Theorem to Secure Quantum Key Distribution," *Nature* **549**, 213 (2007). SigniQ · Team Prometheus · SIH26141 · classical software simulation — no quantum hardware
7. B. Hensen et al., "Loophole-Free Bell Inequality Violation..." *Nature* **526**, 682 (2015).
8. W.-Y. Hwang, "Quantum Key Distribution with High Loss..." *Phys. Rev. Lett.* **91**, 057901 (2003).
9. H.-K. Lo, X. Ma, K. Chen, "Decoy State Quantum Key Distribution," *Phys. Rev. Lett.* **94**, 230504 (2005).
10. W. Hoeffding, "Probability Inequalities for Sums of Bounded Random Variables," *JASA* **58**, 13 (1963).
11. H. Chernoff, "A Measure of Asymptotic Efficiency for Tests of a Hypothesis Based on the Sum of Observations," *Ann. Math. Stat.* **23** (1952).
12. C. H. Bennett, G. Brassard, C. Crépeau, R. Jozsa, A. Peres, W. K. Wootters, "Teleporting an Unknown Quantum State via Dual Classical and Einstein-Podolsky-Rosen Channels," *Phys. Rev. Lett.* **70**, 1895 (1993).
13. D. Gottesman, I. Chuang, "Quantum Digital Signatures," arXiv:quant-ph/0105032 (2001).
14. P. Wallden, V. Dunjko, A. Kent, E. Andersson, "Quantum Digital Signatures with Quantum Key Distribution," *Phys. Rev. A* **90**, 012304 (2014).
15. A. Weng et al., "Quantum Digital Signature with Six-State States," *npj Quantum Information* (2021). arXiv:2104.12059.
16. B. Laurie, A. Langley, E. Kasper, E. Messeri, R. Stradling, "Certificate Transparency," RFC 6962 / RFC 9162 (IETF).
17. M. Pease, R. Shostak, L. Lamport, "Reaching Agreement in the Presence of Faults," *JACM* **27**(2) (1980).
18. R. Renner, "Security of Quantum Key Distribution," PhD thesis (2005) — finite-key framework.

Companion documents: clause-by-clause deliverables audit (`DELIVERABLES.md`), literature-to-code map (`PAPERS.md`), formal QDS model (`QDS_MATH_MODEL.md`), engineering handbook (`PROJECT_HANDBOOK.md`), dashboard manual (`DASHBOARD_MANUAL.md`).